Technical Debt Report

ExamForge AI | Generated: 2026-08-02

Executive Summary

The technical debt audit identified 109 findings across all audit categories. The debt is categorized into four risk levels: Critical (blocks production deployment), High (must fix before enterprise deployment), Medium (should fix for production quality), and Low (nice to have). The total estimated effort to resolve all Critical and High debt is approximately 15-20 developer-days. The application has a solid architectural foundation, but implementation quality gaps in data scoping, security, performance, and accessibility create significant technical debt that must be addressed before production deployment.

Debt Summary by Category

Category	Critical	High	Medium	Low	Total
Code Quality	2	6	7	3	18
Security	2	4	4	2	12
Performance	3	8	10	6	27
Accessibility	4	12	14	6	36
Data Integrity	8	8	6	4	26
Realtime/Offline	4	5	5	3	17
Total	23	43	46	24	136

Debt by Type

Architecture Debt

The application has 4 dead architecture paths: Prisma ORM, TanStack Query, Dexie offline storage, and next-pwa. These are installed and partially configured but never used. They represent both bundle bloat and maintenance burden. The recommended approach is to either fully adopt each technology (implementing the missing integration) or remove it entirely. The estimated effort to adopt TanStack Query is 2-3 days, Dexie is 5-7 days, and removing all dead paths is 1 day.

Security Debt

The most significant security debt is the absence of Content Security Policy headers and the lack of authorization checks on server actions. These are not difficult to fix (estimated 2-3 days total) but are critical for production deployment. The auth callback open redirect is a simple fix (1 hour). Rate limiting requires infrastructure decisions (Redis vs. in-memory) and is estimated at 2-3 days.

Data Debt

The data scoping bugs (school admins seeing cross-school data, teachers seeing all pending grading) are the most critical data debt. These are not complex to fix (estimated 1-2 days) but are data security issues that must be resolved. The N+1 query patterns in the reports service require

more significant refactoring (estimated 3-4 days). The unbounded queries need pagination implementation (estimated 2-3 days). The analytics date range filter is a simple fix (1 hour).

Performance Debt

The most impactful performance debt is the app layout being a client component. Fixing this requires splitting the layout into a Server Component wrapper and a Client Component shell, which is estimated at 2-3 days. The analytics and notifications pages should be refactored to Server Components (2-3 days). Dynamic importing of recharts and removing unused packages is straightforward (1 day). Adding per-route loading.tsx files is estimated at 1-2 days.

Accessibility Debt

The accessibility debt is the most extensive category (36 findings). The most impactful fixes are: making data table sort headers keyboard-accessible (1 day), adding skip-link to the authenticated layout (2 hours), adding prefers-reduced-motion support (1 day), and adding ARIA labels to all interactive elements (2-3 days). The total estimated effort for accessibility is 5-7 days.

Realtime/Offline Debt

The realtime debt includes duplicate subscriptions, no onAuthStateChange listener, and no reconnection data reconciliation. These are estimated at 2-3 days to fix. The offline debt is more significant: implementing Dexie-based offline storage, mutation queue, network recovery, and conflict resolution is estimated at 7-10 days. The offline debt may be deferred if the application does not require offline CBT functionality immediately.

Effort Estimation

Priority	Category	Estimated Effort	Risk if Deferred
P0	Data scoping bugs	1-2 days	Data security breach
P0	Middleware RBAC fix	0.5 days	No role-based access control
P0	CSP + server action auth	2-3 days	XSS, unauthorized modifications
P0	Analytics date filter fix	1 hour	Incorrect analytics
P0	Teacher dashboard fix	1 hour	Incorrect business metrics
P0	Auth callback redirect fix	1 hour	Phishing vulnerability
P0	Pagination for list queries	2-3 days	Timeouts at scale
P1	N+1 query refactoring	3-4 days	Timeouts at scale
P1	App layout RSC split	2-3 days	No SSR benefits
P1	Analytics/notifications RSC	2-3 days	No SSR, slow FCP
P1	Remove unused packages	1 day	Bundle bloat, security surface
P1	Dynamic import recharts	0.5 days	~200KB in main bundle
P1	onAuthStateChange + realtime	2-3 days	Session desync
P1	Rate limiting	2-3 days	Brute force, spam
P1	Structured logging	1-2 days	No observability
P1	Accessibility P0 fixes	2-3 days	WCAG violations

Priority	Category	Estimated Effort	Risk if Deferred
P2	Offline support (Dexie)	7-10 days	No offline CBT
P2	TanStack Query adoption	2-3 days	No client-side caching
P2	Dead code removal	1-2 days	Maintenance burden
P2	Accessibility P1/P2 fixes	3-4 days	WCAG AA compliance